TRƯỜNG CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG – ĐHBK HÀ NỘI

BỘ MÔN KHOA HỌC MÁY TÍNH & HỆ THỐNG THÔNG TIN

CẨM NANG HƯỚNG DẪN THỰC HIỆN ĐỒ ÁN 2

Đề tài: Hệ Thống AI Agent Hỗ Trợ Tra Cứu và Quản Lý Vật Tư - Thiết Bị Phòng Ban

---

CHUYÊN ĐỀ TUẦN 02 (DÀNH CHO NGƯỜI MỚI BẮT ĐẦU TỪ CON SỐ 0)

LẬP TRÌNH PYTHON NỀN TẢNG, KẾT NỐI SQLITE & SQLALCHEMY ORM

Sinh viên thực hiện: Trần Huy Hoàng | **MSSV:** 20237337

Đơn vị áp dụng thử nghiệm: Phòng Hành chính – Quản trị (Ban Quản lý Cơ sở vật chất)

Thời gian học tập: Tuần 2 (Xây Dựng Tầng Dữ Liệu & Thao Tác CSDL Bằng Code)

---

1. Mục Tiêu Tuần 02 & Đánh Giá Bài Tập Tuần 01

Chào Hoàng, thầy rất vui vì em đã nắm vững cấu trúc 4 bảng CSDL ở Tuần 01. Nhớ lại câu hỏi tuần trước: *Tại sao cần tách bảng* ***knowledge_docs****?*, em đã hiểu chuẩn hóa dữ liệu giúp hệ thống không bị trùng lặp thông tin cẩm nang khi công ty có nhiều chiếc máy cùng một model.

Mục tiêu trọng tâm của Tuần 02 là: **Biến bản vẽ thiết kế trên giấy thành các dòng code Python thực sự chạy được trên máy tính**:

1. Hiểu môi trường ảo (`.venv`) là gì và tại sao lập trình viên bắt buộc phải dùng nó.
2. Hiểu bản chất của **ORM (Object-Relational Mapping)**: Tại sao ta không viết câu lệnh SQL bằng chuỗi ký tự mà dùng Class Python.
3. Tự tay viết code kết nối SQLite, định nghĩa các Model và nạp dữ liệu mẫu ban đầu qua script `init_db.py`.

2. Kiến Thức Căn Bản: Môi Trường Ảo (Virtual Environment)

Nhiều sinh viên mới học hay mở Terminal lên và gõ ngay lệnh `pip install sqlalchemy`. Khi về nhà hoặc đổi máy tính khác, code bị lỗi tung tóe vì máy thiếu thư viện hoặc phiên bản thư viện bị xung đột.

Môi trường ảo (`.venv`) là gì?

Môi trường ảo là một **"chiếc hộp cách ly"** nằm ngay bên trong thư mục dự án của em. Khi cài đặt bất kỳ thư viện nào (như SQLAlchemy, Streamlit, LangGraph), thư viện đó chỉ nằm trong thư mục `.venv/` của đồ án này, không ảnh hưởng đến hệ điều hành và các phần mềm khác trên máy tính.

Lệnh chuẩn để khởi tạo và kích hoạt môi trường ảo:

```
# 1. Tao moi truong ao bang Python 3.11 hoac cong cu uv
uv venv --python 3.11 .venv
# 2. Kich hoat moi truong ao
source .venv/bin/activate        # Tren Linux / macOS
.venv\Scripts\activate            # Tren Windows Command Prompt
# 3. Cai dat thu vien tu file requirements.txt
pip install -r requirements.txt
```

3. Bản Chất Của SQLAlchemy ORM: "Thông Dịch Viên" Giữa Python & CSDL

3.1. Tại sao không viết câu lệnh SQL thuần túy?

Bình thường, nếu muốn lấy thông tin của món giấy A4, ta phải viết:

```
SELECT * FROM assets WHERE code = 'VT-GIAY-A4';
```

Cách làm này có 2 nhược điểm chết người:

- **Dễ lỗi gõ phím:** Câu lệnh là chuỗi ký tự (String), nếu gõ sai tên cột thì chương trình chỉ báo lỗi khi đang chạy thật.
- **Nguy cơ SQL Injection:** Kẻ xấu có thể nhập câu lệnh ác ý vào ô chat để phá hủy CSDL.

3.2. ORM giải quyết bài toán này như thế nào?

ORM (Object-Relational Mapping) là một kỹ thuật biến:

- Mỗi **Bảng dữ liệu** trong SQLite → thành một **Class (Lớp)** trong Python.
- Mỗi **Cột** trong bảng → thành một **Thuộc tính (Attribute)** của Class.
- Mỗi **Dòng dữ liệu** → thành một **Đối tượng (Object)** thực sự trong bộ nhớ Python.

Thay vì viết câu SQL bằng chữ, em chỉ cần viết:

```
asset = db.query(Asset).filter(Asset.code == "VT-GIAY-A4").first()
print(f"Ten vat tu: {asset.name}, So luong con: {asset.quantity}")
```

4. Mổ Xẻ Chi Tiết Mã Nguồn Tầng Dữ Liệu

4.1. 1. File 'src/core/database.py' – Cấu hình kết nối CSDL

Hãy mở file `src/core/database.py`, em sẽ thấy các dòng lệnh sau:

```
from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker, declarative_base
from src.core.config import DATABASE_URL

# Khoi tao engine ket noi SQLite
engine = create_engine(
    DATABASE_URL,
    connect_args={"check_same_thread": False}, # CO QUAN TRONG!
    echo=False
)

# Factory tao phien lam viec (Session)
SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)
```

```
14
15 # Lop cha cua toan bo cac Model
16 Base = declarative_base()
```

Tại sao tham số `check_same_thread=False` là bắt buộc?

Mặc định SQLite chỉ cho phép 1 luồng xử lý (thread) truy cập vào file CSDL. Nhưng ứng dụng Web Streamlit ở các tuần sau hoạt động theo mô hình đa luồng (multi-threaded) khi có nhiều sự kiện người dùng bấm chuột. Tham số này cho phép nhiều luồng cùng chia sẻ kết nối SQLite an toàn mà không bị lỗi *"SQLite objects created in a thread can only be used in that same thread"*.

4.2. 2. File 'src/models/asset.py' – Định nghĩa bảng Vật tư & Cẩm nang

Dưới đây là cách chuyển thiết kế CSDL của Tuần 1 thành Class Python:

```
1  from datetime import datetime
2  from sqlalchemy import Column, Integer, String, Text, DateTime
3  from src.core.database import Base
4
5  class Asset(Base):
6      __tablename__ = "assets"
7
8      id = Column(Integer, primary_key=True, autoincrement=True)
9      code = Column(String(50), unique=True, nullable=False, index=True)
10     name = Column(String(200), nullable=False)
11     category = Column(String(50), nullable=False)  # 'Vat tu tieu hao' / 'Thiet bi
           dung chung'
12     model = Column(String(100), nullable=True, index=True)
13     quantity = Column(Integer, default=0, nullable=False)
14     unit = Column(String(20), nullable=False)       # Ram, Chiec, Hop...
15     unit_price = Column(Integer, default=0)          # VND
16     location = Column(String(100), default="Kho Tang 1")
17     status = Column(String(50), default="San sang")
18     aliases = Column(String(255), nullable=True)    # Danh sach bi danh doi thuong
19     specs = Column(String(500), nullable=True)
20     created_at = Column(DateTime, default=datetime.utcnow)
21
22     def __repr__(self):
23         return f"<Asset({self.code}: {self.name} - {self.quantity} {self.unit})>"
```

4.3. 3. Thao tác CRUD (Create, Read, Update, Delete) căn bản

Để làm việc với CSDL trong code, em luôn tuân theo quy trình 3 bước:

1. **Mở phiên (Open Session):** `db = SessionLocal()`
2. **Thực hiện thao tác (CRUD):** Thêm, đọc, sửa, xóa
3. **Lưu và đóng phiên (Commit & Close):** `db.commit(); db.close()`

Ví dụ 1: Thêm một món hàng mới vào kho (Create)

```
1 db = SessionLocal()
2 new_paper = Asset(
3     code="VT-GIAY-A4",
4     name="Giay in Double A A4 70gsm",
```

```
    category="Vat tu tieu hao",
    quantity=85,
    unit="Ram",
    unit_price=75000,
    aliases="giay in, giay a4, double a, ram giay"
)
db.add(new_paper)
db.commit()  # Luu chinh thuc vao o dia
db.close()
```

Ví dụ 2: Xuất kho an toàn chống âm kho (Update with Business Logic)

```
def export_asset(asset_code: str, request_qty: int):
    db = SessionLocal()
    try:
        asset = db.query(Asset).filter(Asset.code == asset_code).first()
        if not asset:
            return "Khong tim thay vat tu!"

        # Kiem tra logic chan am kho
        if asset.quantity < request_qty:
            return f"Kho khong du! Hien chi con {asset.quantity} {asset.unit}."

        asset.quantity -= request_qty  # Tru so luong
        db.commit()                    # Luu lai
        return f"Xuat kho thanh cong! Con lai: {asset.quantity} {asset.unit}."
    finally:
        db.close()
```

5. Bài Tập Thực Hành Tuần 02 Dành Cho Sinh Viên

Hoàng hãy mở file `tests/test_system.py` và thử viết một hàm kiểm tra đơn giản theo 2 yêu cầu sau:

1. **Bài tập 1 (Truy vấn lọc):** Viết một đoạn code Python sử dụng SQLAlchemy để tìm tất cả các mặt hàng thuộc nhóm *"Vật tư tiêu hao"* có số lượng tồn kho ≤ 10 đơn vị (các mặt hàng sắp hết cần cảnh báo).
2. **Bài tập 2 (Xử lý Transaction):** Nếu trong lúc xuất kho, máy tính bị mất điện đột ngột giữa chừng trước khi gọi lệnh `db.commit()`, thì số lượng trong file SQLite có bị trừ mất không? Tại sao? (Gợi ý: Tìm hiểu tính chất **Atomicity (Nguyên tử)** trong giao dịch ACID).

Gợi Ý Đáp Án Bài Tập 2

Nhờ tính chất Atomicity của giao dịch ACID trong SQLite/SQLAlchemy: Một giao dịch chỉ được ghi nhận khi nó hoàn tất trọn vẹn 100%. Nếu mất điện trước lệnh `db.commit()`, toàn bộ thao tác trong phiên làm việc sẽ tự động bị hủy bỏ (**Rollback**) về trạng thái ban đầu, số lượng tồn kho trong file CSDL không bao giờ bị sai lệch!

Thầy chúc mừng em đã hoàn thành Tuần 02! Bây giờ em đã có trong tay CSDL SQLite hoạt động trơn tru. Ở Tuần 03, chúng ta sẽ bắt đầu bước vào thế giới kỳ thú của **AI Agent và LangGraph**!